# Baseline Analysis

## Table of Contents


Initial Architecture....2
Communication between components....2
An Overview on the cryptography mechanisms....2
The Lack of features....2
The Sensor....3
pre_shared_key : Bytes....3
data_buffer : List....3
generateData() : String....3
encryptData(data: String) : Bytes....3
sendData(data: String) : Bytes....3
The Edge Gateway....4
pre_shared_key : Bytes....4
p : Integer....4
g : Integer....4
dh_private_key : Integer....4
dh_public_key : Integer....4
shared_secret : Integer....4
cloud_aes_key : Bytes....5
negotiateSession(cloud_service : CloudService) : void....5
receiveData(legacy_device : LegacyDevice) : Bytes....5
decryptData(encryptedData : Bytes) : String....5
reEncryptData(plaintext : String) : Bytes....5
forwardData(cloud_service : CloudService, reEncryptedData : Bytes) : void....5
The Cloud Service....6
p : Integer....6
g : integer....6
dh_private-key : Integer....6
dh_public_key : Integer....6
shared_secret : Integer....6
aes_key : Bytes....6
incoming_buffer : List....7
negotiate_dh(gatewaypublic_key : Integer) : Integer....7
receiveData(encrypted_data : Bytes) : void....7
ProcessData() : String....7
Sequence Diagram – Baseline System....8
Baseline Test and Measurements....9
Cryptography unit test....9
End-To-End Integration Test....9
Performance Metric....9

# Initial Architecture

The Legacy system consist of three main components:

- The Sensor (Represented as “Legacy Device”).
- The Edge Gateway.
- The Cloud Service

## Communication between components

In order to keep the system as simple as possible the communication between the devices does not happen via a real TCP connection. In this initial architecture the communication between the devices is implemented via buffers: a sender (e.g. the Edge Gateway) when need to send data to receiver (e.g. the Cloud Service) just put the data into a buffer that follows a FIFO policy (usually implemented as a List object), the receiver is then responsible for retrieving the data from that buffer.

## An Overview on the cryptography mechanisms

In order to simulate the strict hardware and power constraints typical of legacy IoT sensors, the Sensor have a hard-coded AES symmetric key shared with the Edge Gateway. The Sensor encrypts its data with this static key and transmits it to the Gateway. The Edge Gateway is then responsible for decrypting it and enciphering it again with another key that is established together with the Cloud service thanks to a well-known key-agreement protocol: The Diffie-Hellman protocol. Since this protocol is vulnerable to quantum attacks, it will be substituted with the ML-KEM in future phases of the project.

## The lack of features

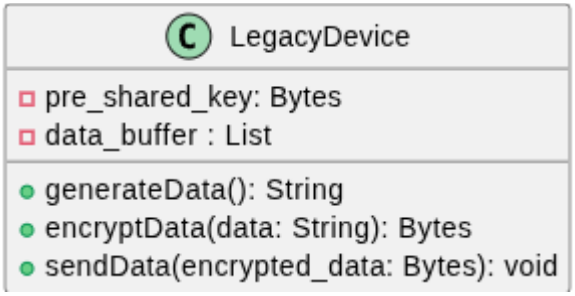
Since the first part of the project focuses on modernizing the key agreement mechanism between the Cloud service and the Edge Gateway, extra functionalities such as databases and GUI to visualize the data stored in the Cloud have not been implemented intentionally.

---

With this system, that aims to mimic a wireless connection, and lacks some advanced functionality, we are both able to start with a simple architecture to understand and leave room for future improvements.

Here we describe the detailed architecture of each component:

## The Sensor




The sensor is the component of the system responsible for capturing data from the environment and sending it to the Edge Gateway.

Here we present a breakdown of its attributes and methods:

### pre_shared_key : Bytes

This data is a hard-coded AES cryptography key that the sensor uses to cipher the data before sending it to the edge gateway. These attributes resemble how legacy IOT device works: not enough computing power to do key-exchange, and then a symmetric key is hard coded into them.

### data_buffer : List

A data structure that aims to represent a small buffer in which the Sensors hold recorded data before sending them to the edge gateway

### generateData() : String

A method to mimic the functionality of the sensor to record data from its surroundings. In the current implementation this is a method that just generates a random string of data followed by the timestamp.

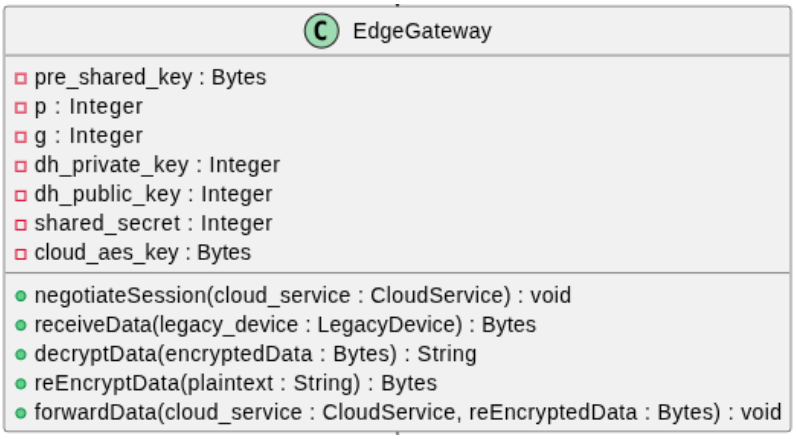
### encryptData(data: String) : Bytes

The AES-encryption primitive, responsible for encrypting the generated data with the pre_shared_key.

### sendData(data: String) : Bytes

This method just put the encrypted data into the data_buffer data structure in order to simulate the “sending” to the edge gateway.

# The Edge Gateway




The Edge Gateway is the core of the system. It is the one in charge of the communication between the Sensor and the Cloud.

Here we present a breakdown of its attributes and methods:

## pre_shared_key : Bytes

The exact same hard-coded AES cryptography key that the Sensor has, the gateway uses it to decrypt the data received from the Sensor.

## p : Integer

The public modulo that will be used in the Diffie-Hellman key exchange mechanism between the Edge Gateway and the Cloud Service.

## g : Integer

The public base that will be used in the Diffie-Hellman key exchange mechanism between the Edge Gateway and the Cloud Service.

## dh_private_key : Integer

The ephemeral private key chosen by the Edge Gateway each time a communication session is initiated.

## dh_public_key : Integer

It is the result of $(g)^{dh_private_key}$ (mod p) that is sent to the Cloud Service during the key agreement procedure.

## shared_secret : Integer

It is the result of modular exponentiation, raising the received public key to the power of its own private key, modulo p.

### cloud_aes_key : Bytes

The symmetric key shared between the Edge Gateway and the Cloud Service, used to cipher the messages directed to the Cloud, it is derived via a single application of the algorithm SHA256 to the shared secret.

### negotiateSession(cloud_service : CloudService) : void

Initiate and complete the negotiation session of the shared secret with the Cloud Service.

### receiveData(legacy_device : LegacyDevice) : Bytes

Method used to simulate receiving data from the Legacy Sensor. It consist into a simple fetch of data from the Sensor Buffer.

### decryptData(encryptedData : Bytes) : String

Method used to decrypt the data received from the sensor with the hard-coded AES symmetric key.

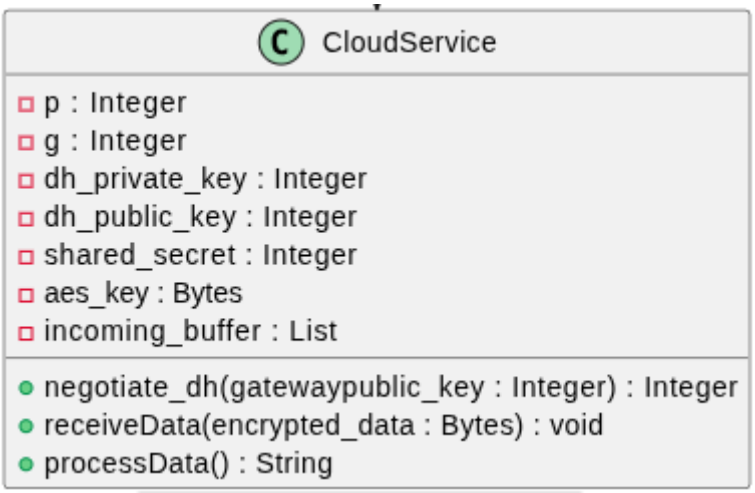
### reEncryptData(plaintext : String) : Bytes

Method used to encrypt data with the shared secret established with the Cloud Service. The algorithm used here is also AES.

### forwardData(cloud_service : CloudService, reEncryptedData : Bytes) : void

Analogous to LegacyDevice.sendData(), but for sending data to the Cloud Service.

## The Cloud Service




The Cloud Service represents the backend infrastructure. It acts as the termination point for the communication with the Edge Gateway and is responsible for final data processing.

Here we present a breakdown of its attributes and methods:

### p : Integer

The public modulo that will be used in the Diffie-Hellman key exchange mechanism between the Edge Gateway and the Cloud Service.

### g : integer

The public base that will be used in the Diffie-Hellman key exchange mechanism between the Edge Gateway and the Cloud Service.

### dh_private_key : Integer

The ephemeral private key chosen by the Cloud Service each time a communication session is initiated.

### dh_public_key : Integer

It is the result of $(g)^{dh_private_key}$ (mod p) that is sent to the Edge Gateway during the key agreement procedure.

### shared_secret : Integer

It is the result of modular exponentiation, raising the received public key to the power of its own private key, modulo p.

### aes_key : Bytes

The symmetric key shared between the Edge Gateway and the Cloud Service, used to cipher the messages direct to the Cloud, it is derived via a single application of the algorithm SHA256 to the shared secret.

### incoming_buffer : List

A FIFO data structure simulating a receiving network socket where the Edge Gateway deposits encrypted payloads.

### negotiate_dh(gateway_public_key : Integer) : Integer

This method is called by the gateway to complete the handshake. It generates the cloud's ephemeral keys, computes the shared AES secret using the gateway's public key, and returns the cloud's public key to the gateway.

### receiveData(encrypted_data : Bytes) : void

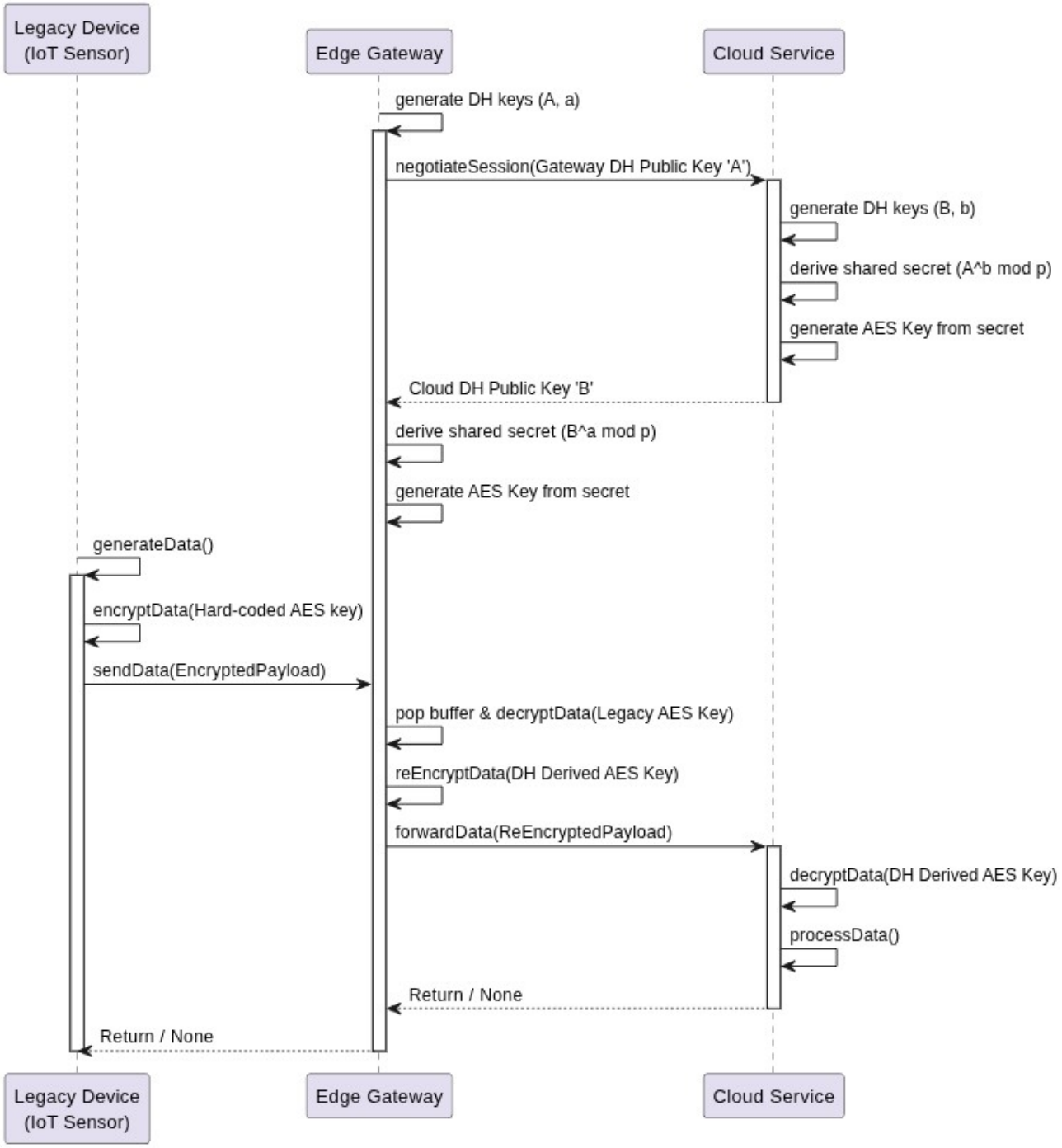
Method used to simulate the reception of data from the Edge Gateway.

### processData() : String

Method used to simulate the processing of data, used to simply decrypt the received data from the Edge Gateway.

# Sequence Diagram – Baseline System

The following diagram shows how the system behaves when a data entry is generated from the sensor. This simple communication involves a single message that travels from the Sensor to the Cloud Service:



# Baseline Test and Measurements

Before introducing the post quantum cryptography key exchange and any other feature in the system, it is necessary to ensure that the legacy system functions correctly. Moreover it is necessary to provide some metrics in order to compare the impact of the future ML-KEM integration with respect to the current key-exchange mechanism.

Then we will proceed with the following test strategy concerning the baseline architecture:

## Cryptography unit test

We aim to verify that the data encrypted by the Legacy Sensor using the hard-coded AES-CBC key can be successfully and deterministically decrypted by the Edge Gateway.

We aim to verify the mathematical correctness of the Edge Gateway and Cloud Service handshake, asserting that both parties independently derive the exact same shared secret and AES session key without transmitting it over the network.

## End-To-End Integration Test

This test will aim to verify the correctness of the initial pipeline, i.e. it will simulate the entire operational flow. It asserts that a randomly generated string on the Legacy Sensor can traverse the gateway and be successfully retrieved in plaintext by the Cloud Service.

## Performance Metric

To critically evaluate the improvement in terms of metrics derived by the introduction of ML-KEM we will first of all measure the performances in terms of memory and time required by the Diffie-Hellman protocol. The metrics we plan to keep track of, in order to compare the two implementations, are:

- Handshake Latency (ms)
- End-to-End Latency (ms)
- Public Key Size (Bytes)
- Ciphertext Size (Bytes)